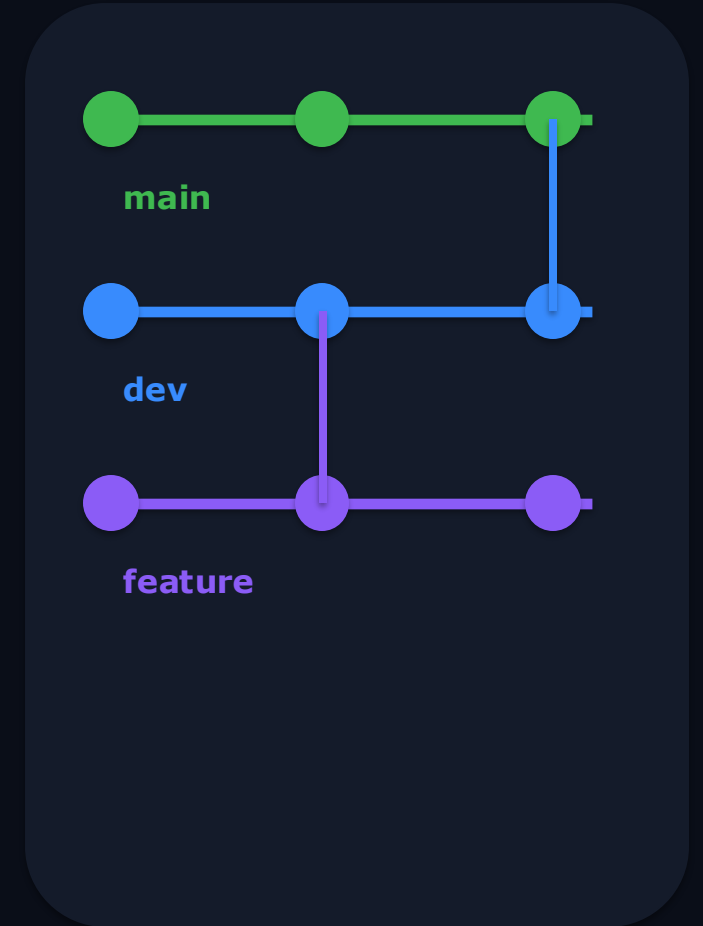


GIT + GITHUB

Do caos de arquivos ao fluxo profissional de desenvolvimento



Roteiro da aula

- 01** O problema antes do Git

- 02** Como o Git pensa e armazena

- 03** Comandos essenciais

- 04** GitHub e trabalho remoto

- 05** Branches, merge e conflitos

- 06** Fluxo profissional + prática

PARTE 1 — Por que o Git existe?

Antes de aprender comandos, entenda o problema que ele resolve.

Antes do Git: “final”, “final2”, “agora-vai”

 projeto.zip

 projeto_final.zip

 projeto_final_v2.zip

 projeto_final_v2_CORRETO.zip

 projeto_final_v2_CORRETO_agora.zip

O que dava errado?

Quem alterou o quê?

Qual arquivo é o mais novo?

Como voltar para ontem?

Como juntar trabalho de 3 pessoas?

Como saber por que uma linha mudou?

O Git resolve 5 dores centrais

HISTÓRICO

Cada mudança importante vira um ponto recuperável.

COLABORAÇÃO

Várias pessoas trabalham sem sobrescrever tudo.

RASTREABILIDADE

Autor, data e motivo ficam ligados à mudança.

EXPERIMENTAÇÃO

Branches permitem testar ideias com isolamento.

RECUPERAÇÃO

Voltar para versões anteriores deixa de ser pânico.

Git ≠ GitHub

GIT

Ferramenta de controle de versão distribuído.

Roda localmente. Cria commits, branches, merges e histórico.

GITHUB

Plataforma para hospedar e colaborar em repositórios Git.

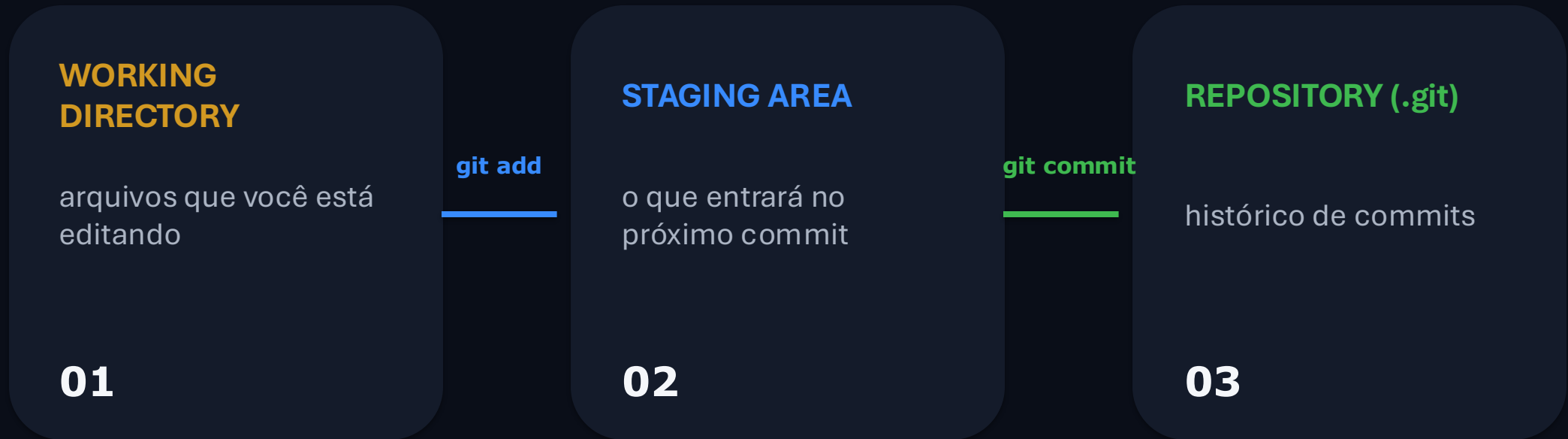
Adiciona Pull Requests, Issues, Actions, permissões e interface web.

Git é o motor. GitHub é uma das plataformas construídas ao redor dele.

PARTE 2 — Como o Git pensa

Snapshot, commit, repositório local e remoto.

As 3 áreas que você precisa enxergar



Commit: um checkpoint com identidade



Snapshot estado dos arquivos naquele momento

Autor quem criou a mudança

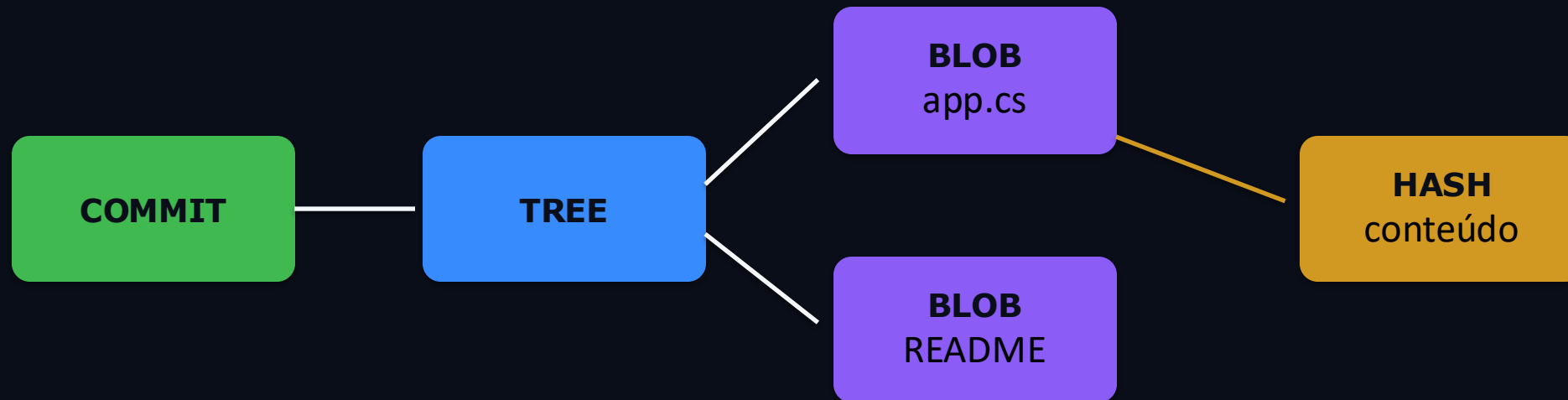
Timestamp quando aconteceu

Parent qual commit veio antes

Hash identificador único do commit

Como os arquivos ficam salvos no “servidor Git”

O Git não guarda simplesmente “pastas de versões”. Ele armazena objetos endereçados por hash.



No GitHub, o repositório remoto recebe esses objetos + referências (branches/tags).

Local x remoto: duas cópias, um histórico sincronizado



PARTE 3 — Comandos essenciais

O fluxo diário do desenvolvedor, do zero ao push.

Começando um repositório



novo projeto

```
$ mkdir meu-projeto  
$ cd meu-projeto  
$ git init  
Initialized empty Git repository
```

init = cria o .git aqui



projeto existente

```
$ git clone https://github.com/org/projeto.git  
Cloning into 'projeto'...  
$ cd projeto
```

clone = baixa repositório + histórico

O ciclo que você mais vai repetir

1

git status

Veja o que mudou

2

git add .

Selecione mudanças

3

git commit -m "..."

Crie o checkpoint

Dica: commit pequeno + mensagem clara = histórico útil.

Entendendo o que aconteceu



histórico

```
$ git log --oneline --graph --all  
* 9f31a7c feat: adiciona checkout  
* 5ab91de fix: valida cupom  
* 2c10e42 chore: estrutura inicial
```

git diff

Mostra diferenças ainda não commitadas.

git show <hash>

Abre detalhes de um commit específico.

Sincronizando com o GitHub

git fetch

Baixa referências do remoto sem misturar no seu trabalho.

git pull

Busca e integra mudanças na branch atual.

git push

Envia seus commits locais para o remoto.

Regra prática: antes de começar, atualize; depois de um conjunto coerente de commits, envie.

Cheat sheet: comandos para a primeira semana

git status

estado atual

git add <arquivo>

prepara arquivo

git commit -m

salva checkpoint

git log --oneline

histórico

git diff

diferenças

git branch

lista branches

git switch -c

cria branch

git merge

mescla branch

git pull

traz mudanças

git push

envia commits

git restore

descarta mudança

git stash

guarda trabalho temporário

PARTE 4 — GitHub no trabalho

Colaboração, Pull Request, revisão e automação.

Do código local ao time



Pull Request não é um comando Git: é um processo de colaboração da plataforma.

Pull Request: “quero integrar esta mudança”

feature/login → dev

✓ testes passaram

✓ revisão de código

💬 2 comentários

Δ 8 arquivos / +214 -37

Por que usar?

Revisão por pares

Discussão registrada

Checks automáticos

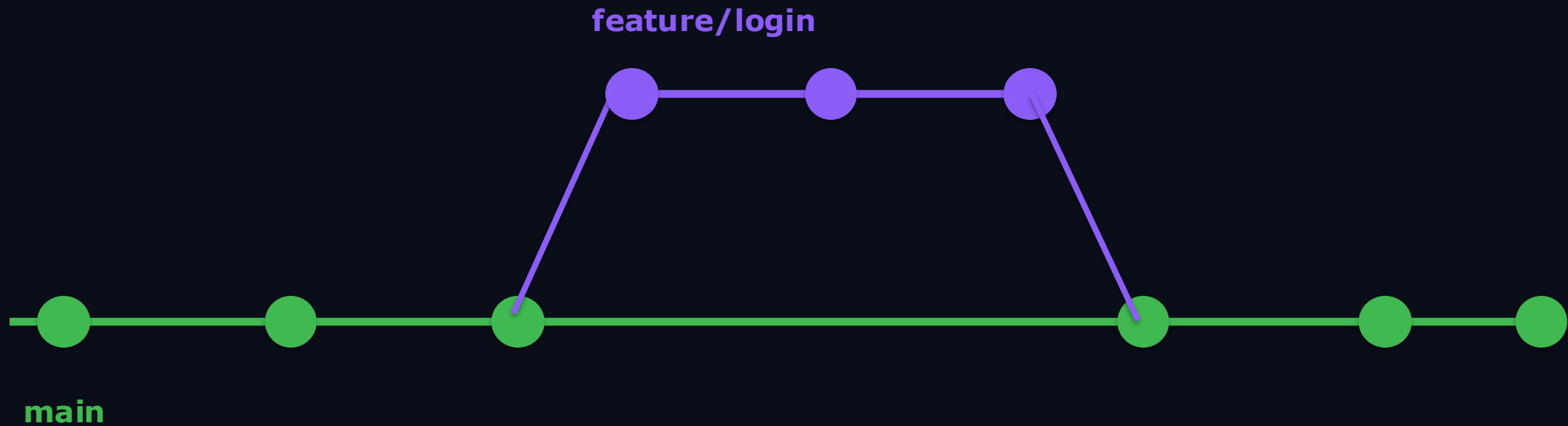
Aprovação antes do merge

Menos risco na branch alvo

PARTE 5 — Branches e Merge

Trabalhar em paralelo sem transformar o repositório em caos.

Branch é uma linha de desenvolvimento



A branch não duplica todo o projeto. Ela é uma referência móvel para commits.

Criando e navegando entre branches



feature

```
$ git switch -c feature/login
```

```
$ git status
```

On branch feature/login

```
$ git add .
```

```
$ git commit -m "feat: login"
```

```
$ git push -u origin feature/login
```

Alternativas

git branch feature/login

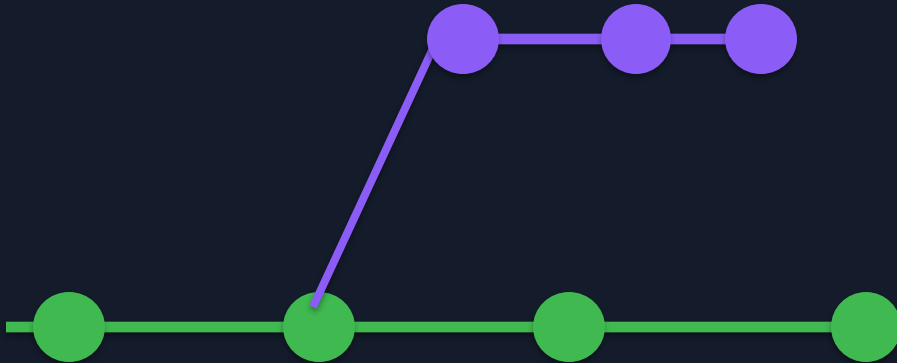
git switch feature/login

git checkout -b ...

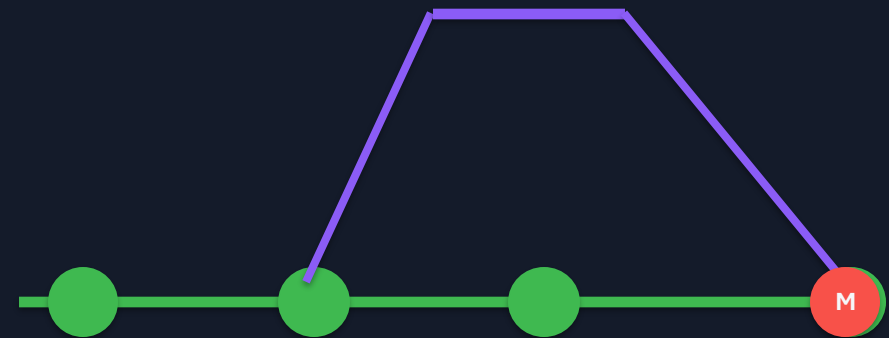
checkout é sintaxe mais antiga e ainda muito comum.

Merge: juntando duas linhas de desenvolvimento

ANTES



DEPOIS



git switch dev → git merge feature/login

Conflito: quando o Git precisa de uma decisão humana



arquivo em conflito

<<<<<< HEAD

preco = 100

=====

preco = 120

>>>>>> feature/nova-regra

Como resolver

1. Entenda as duas intenções
2. Edite o arquivo com a versão correta
3. git add <arquivo>
4. finalize o merge/commit
5. rode os testes

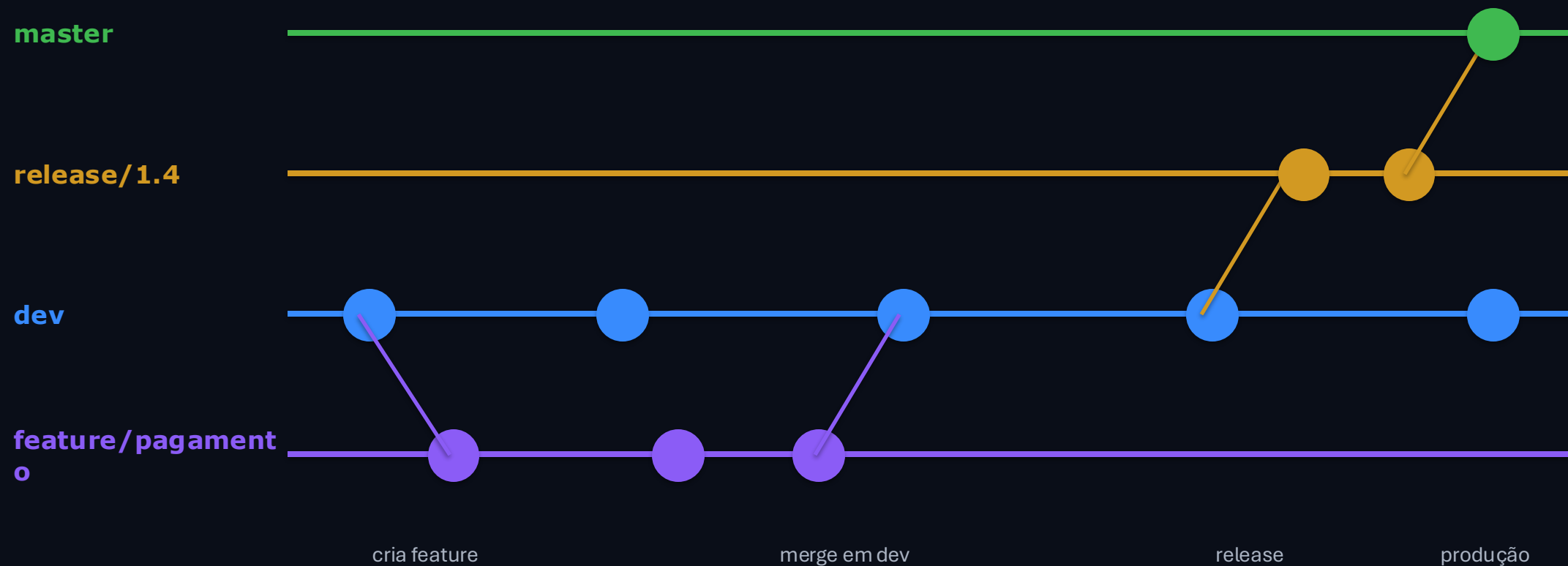
Conflito não é erro do Git. É o Git se recusando a adivinhar.

Um fluxo de branches usado em muitos times

master / main		PRODUÇÃO
release/*		preparação da versão
dev		integração contínua
feature/*		novas funcionalidades
bugfix/*		correções não urgentes
hotfix/*		correção urgente em produção

Importante: “master = produção” é uma convenção do fluxo do time, não uma regra técnica do Git.

Exemplo: uma feature chegando até produção



Hotfix: produção não pode esperar a próxima release



Depois: sincronizar também com dev

Branches e commits que contam uma história

BRANCHES

feature/checkout-pix

bugfix/cupom-expirado

hotfix/login-500

release/2.3.0

COMMITTS

feat: adiciona pagamento PIX

fix: corrige validação do cupom

docs: atualiza README

refactor: simplifica cálculo

Demonstração ao vivo:

1 Criar repositório no GitHub

2 `git clone`

3 Criar README + `git status`

4 `git add` + `git commit`

5 `git push`

6 Criar feature/aula-git

7 Alterar arquivo + novo commit

8 Push da feature

9 Abrir Pull Request

10 Merge para dev/main

11 Mostrar `git log --graph`

Erros comuns de quem está começando

Comitar segredo

senha, token, connection string

Trabalhar direto na master

sem revisão e sem isolamento

Commit gigante

mistura várias intenções

Dar pull sem entender

pode integrar mudanças inesperadas

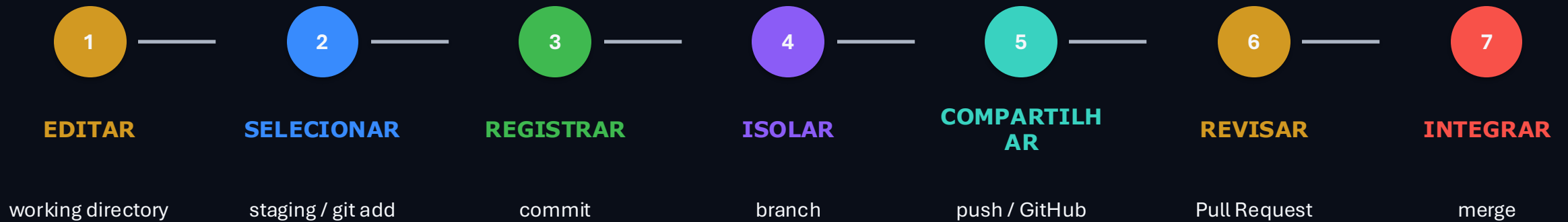
Mensagem “ajustes”

histórico perde valor

Resolver conflito no chute

pode apagar regra importante

O mapa mental para levar da aula



Git é memória. Branch é isolamento. GitHub é colaboração.

PERGUNTAS?

Se você entendeu commit + branch + merge, já tem a base para evoluir rápido.

```
git log --oneline --graph --all
```